AAPE – Autonomous Agents Practice Environment

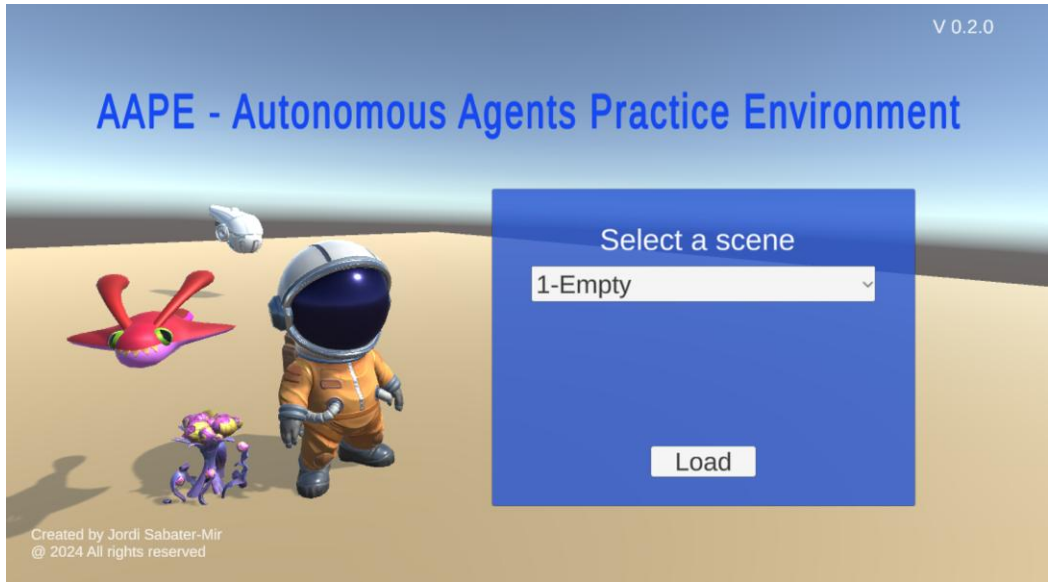
Updated to version 0.4.1

Jordi Sabater-Mir
jsabater@iiia.csic.es

IIIA-CSIC INSTITUT D'INVESTIGACIÓ EN INTEL·LIGÈNCIA ARTIFICIAL

CSIC CONSEJO SUPERIOR DE INVESTIGACIONES CIENTÍFICAS

UAB Universitat Autònoma de Barcelona


V 0.2.0
AAPE - Autonomous Agents Practice Environment
Select a scene
1-Empty
Load
Created by Jordi Sabater-Mir
@ 2024 All rights reserved

Home
1
2
3
4
set
set
set
set
Fr
Top
Critter1
Astronaut1
B
F
T
Settings
Debug mode
Manual control
Send message
RUNNING
STOP
0

Game Engine

- A game engine is a software framework designed to facilitate the creation and development of video games.

- It provides developers with a set of tools, libraries, and features to streamline the game development process.

Serious Games and Edutainment

Simulation and Training

Film and Animation

Architectural Visualization

Virtual Reality (VR)
Augmented Reality (AR)

Components of a Game Engine

- **Rendering Engine:** Handles graphics rendering, including rendering 2D and 3D graphics, lighting, shaders, and special effects.
- **Physics Engine:** Simulates real-world physics, allowing objects to interact with each other and the environment realistically.
- **Audio Engine:** Manages sound effects, music, and other audio elements within the game.
- **Scripting Engine:** Allows developers to write code to define game logic, behaviors, and interactions.
- **Networking Engine:** Facilitates multiplayer functionality, enabling players to connect and interact in online environments.
- **Asset Pipeline:** Streamlines the management and integration of game assets such as textures, models, animations, and sounds.
- **Editor Tools:** Provides a graphical interface for developers to design game levels, set parameters, and test gameplay elements.

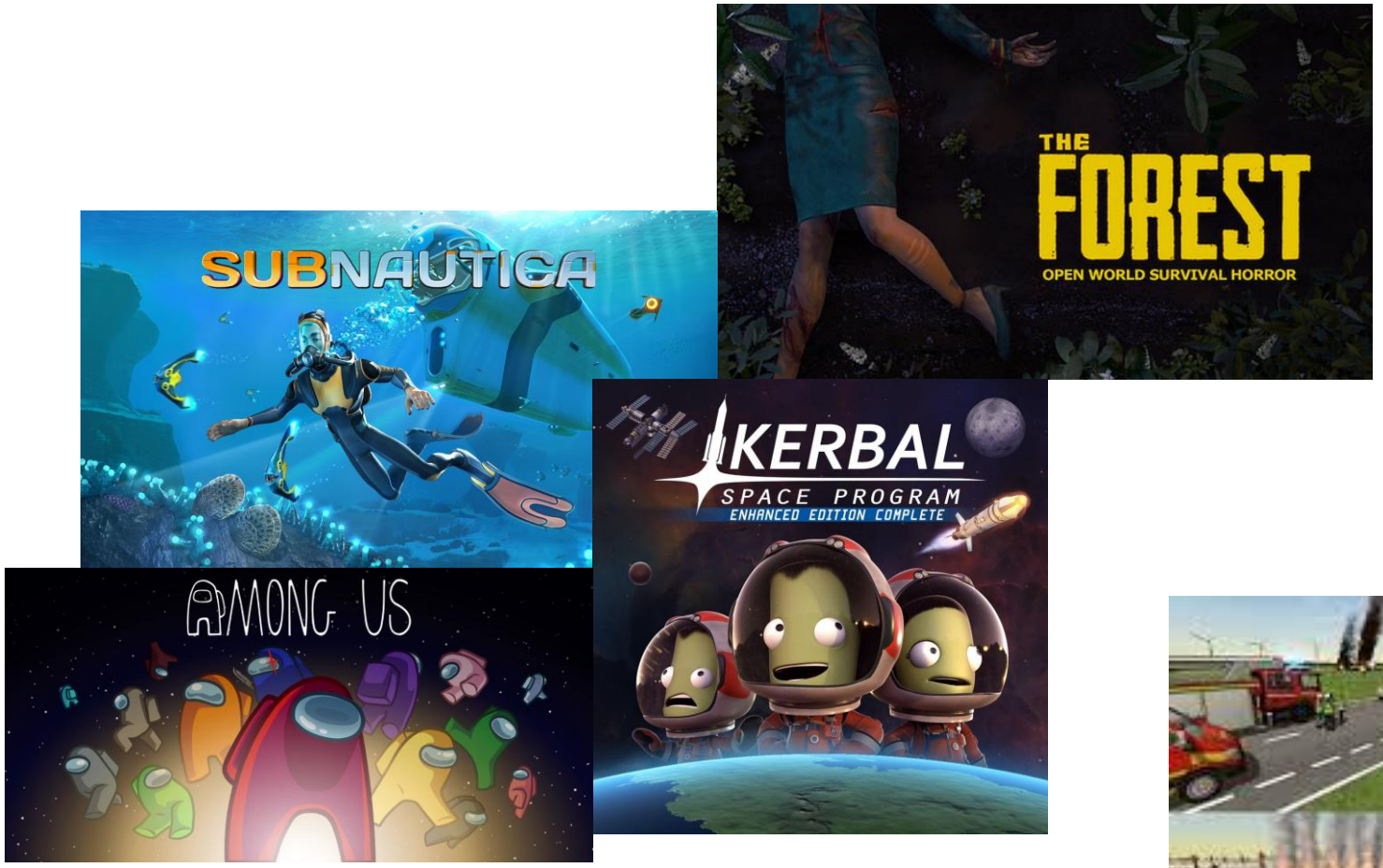
- Unity is a well known game engine used for creating 2D, 3D, VR, and AR experiences.

- It provides a comprehensive set of tools for game development, including rendering, physics, audio, scripting, and more.

- Unity is popular among indie developers, professionals, and hobbyists alike due to its ease of use and robust feature set.

- Cross-Platform Development: Unity supports deployment to various platforms including PC, Mac, Linux, mobile devices (iOS, Android), consoles, and web.

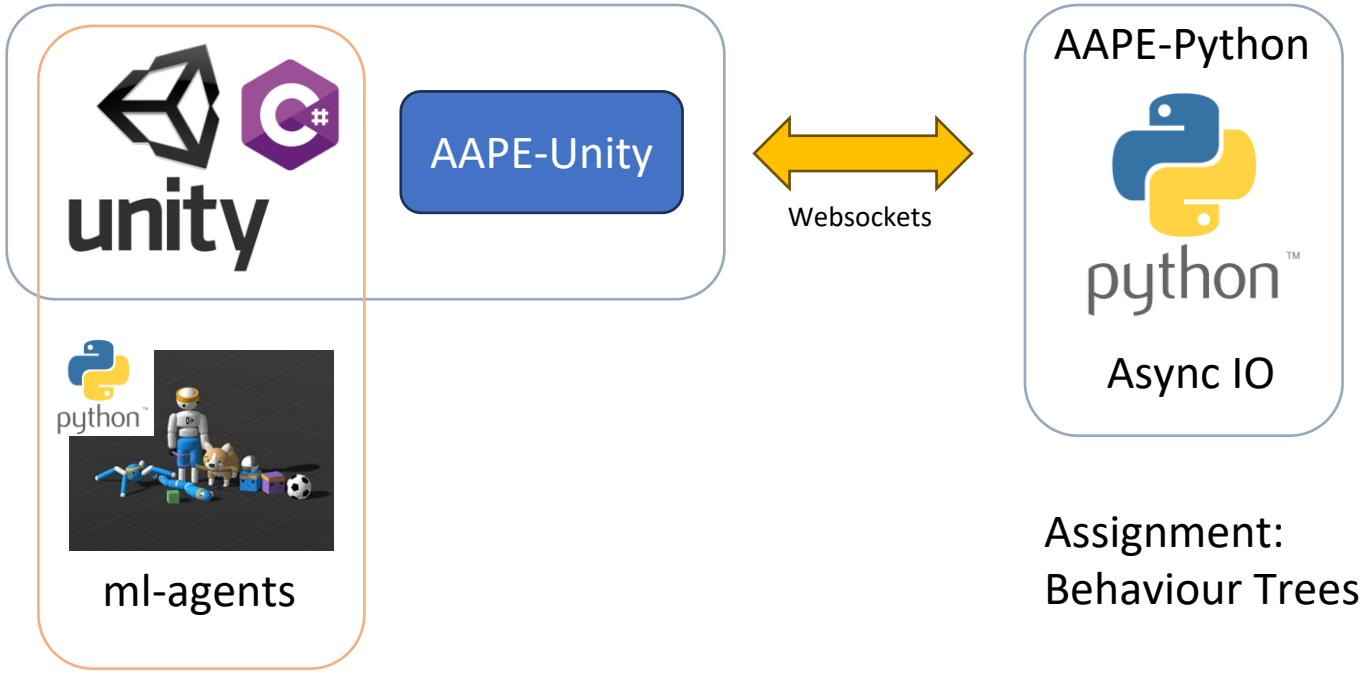
- Scripting: Supports multiple programming languages such as **C#**, JavaScript, and Boo for scripting gameplay logic.


SUBNAUTICA
THE
FOREST
OPEN WORLD SURVIVAL HORROR
KERBAL
SPACE PROGRAM
ENHANCED EDITION COMPLETE
AMONG US

XVR

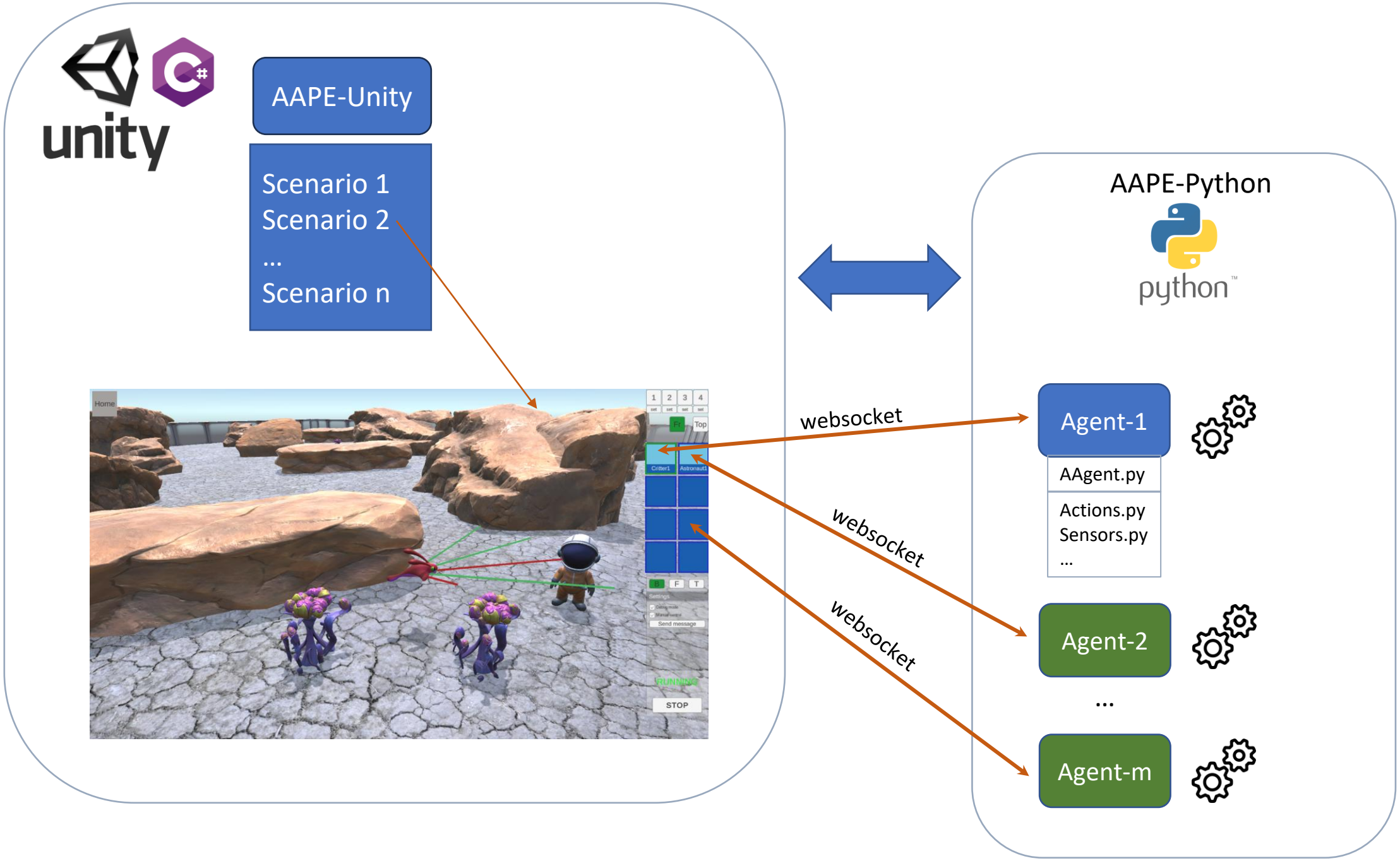
XVR
XVR
XVR
XVR

Elements of the practice environment



Assignment:
Behaviour Trees

Assignments:
Reinforcement Learning


unity
AAPE-Unity
Scenario 1
Scenario 2
...
Scenario n
Home
STOP
websocket
websocket
websocket
AAPE-Python
python
Agent-1
AAgent.py
Actions.py
Sensors.py
...
Agent-2
...
Agent-m

AAPE interface

Return to the scenario selection screen

World cameras

Agent active (green border)

Agent empty slot

Agent follow cameras

Debug mode
Manual control

Send message
To AAPE-Python

Simulation state

Change simulation State
[on hold | running]

Active agent inventory

Start – Stop the simulation

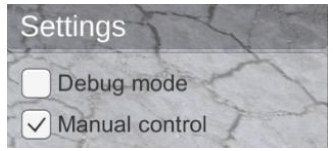
- <u>Starts or puts “on hold” the simulation.</u>

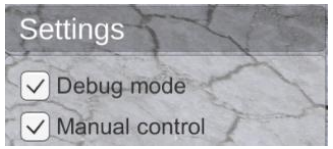
- When **“on hold”**, the agents
  - Do not perform any actions or goals.
  - Messages are neither sent nor processed in AAPE-Unity.
  - Manual control does not work.
  - Python-Unity also enters a hold state (there is an exception that we will see later on when we talk about behavior trees and its associated goals)

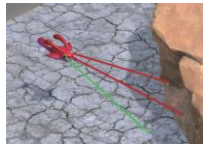
Cameras

- Initially the scenario starts with the top view. As soon as “Fr” is selected it starts showing the Free camera. You can return to the top view pressing “Top”.
- While using the “Fr” camera, pressing the **right mouse button** on the scenario window activates the camera movement and allows to change the view using the **mouse + W-A-S-D**. The **Left-Shift key** increases the speed of the camera movement. **X key** blocks the height so you can move the camera freely without going up and down. Pressing the right mouse button again returns to the UI mode.
- If you “set” one of the slots (1 to 4), the current view of the free camera is stored in that slot. Then you can recover the view when needed by pressing the corresponding number.

- The agent follow camera buttons activate the cameras that follow the agent (“B” - Back, “F” - Front, “T” - Top).







Agent debugging

- "Debug mode" activates the visualization of the ray cast sensor.
- "Manual control" allows to move the agent manually using W-A-S-D.

IMPORTANT: When you press Manual control, if the agent was executing a goal or a behavior tree, it will stop running. When the Manual control is deactivated, the previous automatic behavior is not recovered and the agent remains idle.

Send Message

- The "Send message" button opens a text edit and allows to send a message to the agent (by pressing the "Send" button or pressing [Enter]. It allows to start manually a single action, a simple behavior (goal) or a behavior tree (bt).

```
action:<instruction>[,param1][,param2,...]

goal:<goal_name>

bt:<behaviour_tree_name>
```

Sending a message only works if the simulation is "RUNNING". If the simulation is "ON HOLD", the sent message is lost.

Messages

`action:<instruction>[,param1][,param2,...]`

Translation Movement

mf

Move forward. *ex: action:mf* *ex Code: self.a_agent.send_message("action","mf")*

mb

Move backward. *ex: action:mb*

ntm

No translation movement (stops mf or mb). *ex: action:ntm*

Rotation Movement

tr

Turn right. *ex: action:tr*

tl

Turn left. *ex: action:tl*

nt

No turn (stops tr or tl). *ex: action:nt*

stop

Stop all kind of movement (translation and rotation). *ex: action:stop*

Messages

`action:<instruction>[,param1][,param2,...]`

NaveMesh movement

walk_to,<target_location_name>

Walk to a location using the NaveMesh navigation system. There is a single parameter with a string with the name of the target location.

ex: action:walk_to,Base *ex Code: self.a_agent.send_message("action","walk_to,Base")*

teleport_to,<target_location_name>

Teleports the agent to a location using the NaveMesh navigation system. There is a single parameter with a string with the name of the target location.

ex: action:teleport_to,Base

Inventory management (see "About the inventory" slides)

get,<object_name>,<amount>

Gets an object from a container inventory and adds it to the agent inventory. It has two parameters, the name of the object type (<object_name>) and the amount of objects of that type we want to transfer (<amount>).

ex: action:get,SoylentGreen,1 *ex Code: self.a_agent.send_message("action","get,SoylentGreen,1")*

leave,<object_name>,<amount>

Moves an object from the agent inventory to a container. It has two parameters, the name of the object type (<object_name>) and the amount of objects of that type we want to transfer (<amount>).

ex: action:leave,SoylentGreen,1

Messages

goal:<goal_name> *self.a_agent.send_message("goal","<goal_name>")*

It calls one of the goals specified in the "Goals_BT.py" file.

bt:<behaviour_tree_name> *self.a_agent.send_message("bt","<behavior_tree_name>")*

It calls one of the behavior trees specified in the corresponding python file.

(for exemple, the BTRoam.py behavior tree)

AAgent_BT.py

Important: Before being able to execute a goal or a behavior tree, they have to be declared in AAgent_BT.

```
# Reference to the possible goals the agent can execute
self.goals = {
    "DoNothing": Goals_BT.DoNothing(self),
    "ForwardStop": Goals_BT.ForwardStop(self),
    "ForwardDist": Goals_BT.ForwardDist(self, -1, 5, 10),
}

# Reference to the possible behaviour trees the agent can
execute
self.bts = {
    "BTRoam": BTRoam.BTRoam(self)
}
```

Examples:

goal:ForwardStop

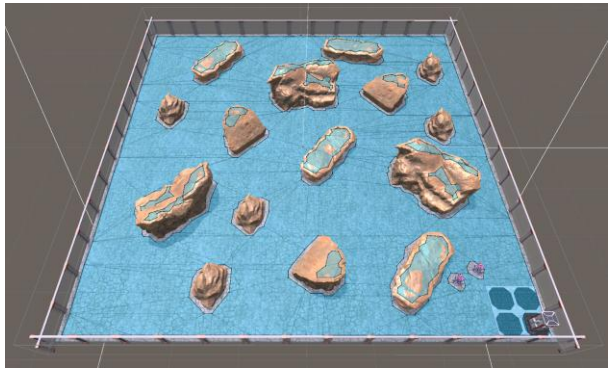
goal:DoNothing

bt:BTRoam

Agent movement

Two forms of movement (one cancels the other):

NavMesh system



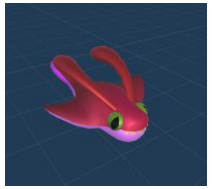
The **NavMesh** (Navigation Mesh) system is a built-in AI pathfinding tool that allows characters to navigate complex environments. It works by generating a walkable surface (mesh) from level geometry, which agents use to calculate obstacle-avoiding paths.

walk_to,<target_location_name>

ex: action:walk_to,Base

teleport_to,<target_location_name>

ex: action:teleport_to,Base



Explicit movement instructions

Translation Movement

mf

Move forward. *ex: action:mf*

mb

Move backward. *ex: action:mb*

ntm

No translation movement (stops mf or mb). *ex: action:ntm*

Rotation Movement

tr

Turn right. *ex: action:tr*

tl

Turn left. *ex: action:tl*

nt

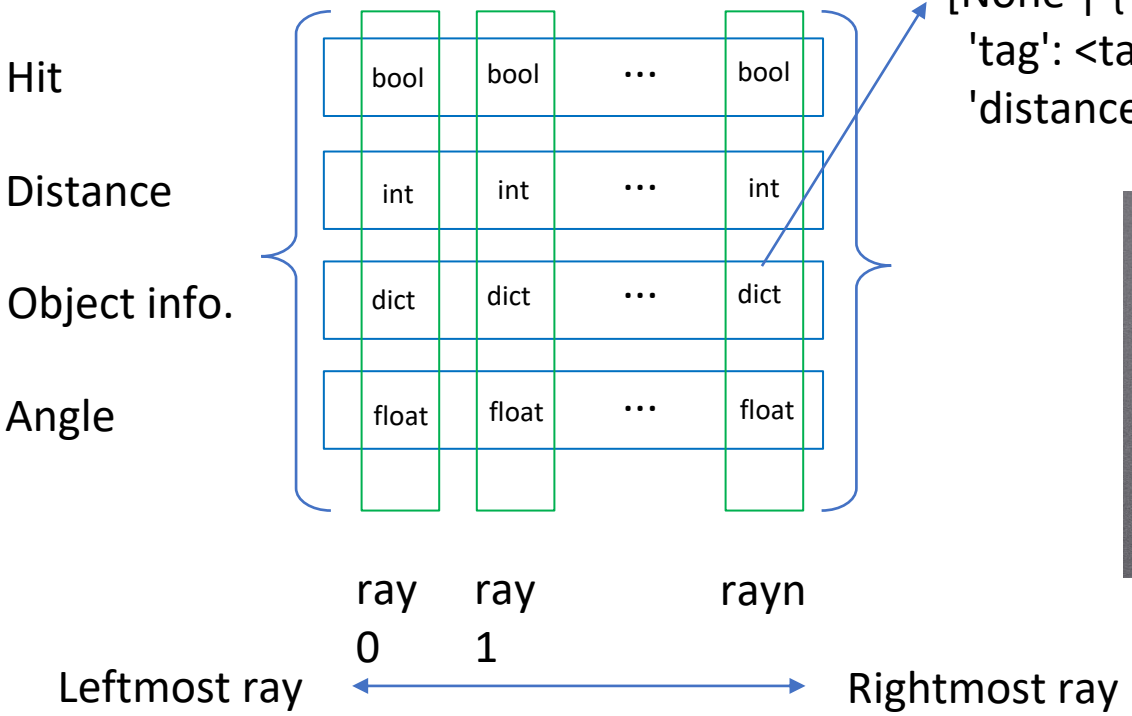
No turn (stops tr or tl). *ex: action:nt*

stop

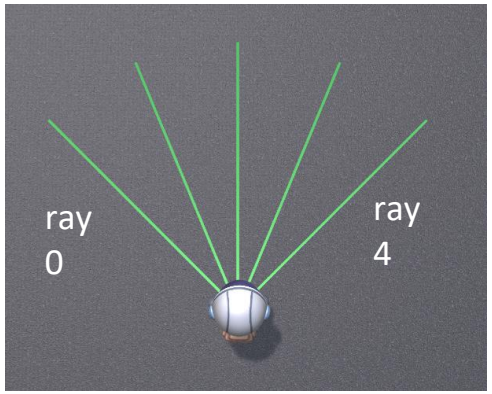
Stop all kind of movement (translation and rotation). *ex: action:stop*

Perception: Class RayCastSensor [Sensors.py]

- You access the information of the ray cast sensor through the property **`self.rc_sensor`**.
- This property is an array where the columns represent each ray cast going from left to right and the rows contain information about (0) if the ray is hitting an object, (1) the distance with the object, (2) information about the object and (3) the angle of the ray with the object.



[None | {'name': <name_of_the_hit_object>,
'tag': <tag_of_the_hit_object>,
'distance': <distance_to_the_hit_object>}]



Angle – Degrees of that ray from the centre. Positive, rays on the right. Negative, rays on the left

Examples:

```
# Iterate the rows of the array
for info_row in self.rc_sensor.sensor_rays:
     print(info_row)
```

[0, 0, 0, 0, 0]
[-1, -1, -1, -1, -1]
[None, None, None, None, None]
[-45.0, -22.5, 0.0, 22.5, 45.0]

```
# Iterate each ray of the sensor
for ray in zip(*self.rc_sensor.sensor_rays):
    print(list(column))
```

[0, -1, None, -45.0]
[0, -1, None, -22.5]
[0, -1, None, 0.0]
[0, -1, None, 22.5]
[0, -1, None, 45.0]

Examples:

```
# Iterate each ray of the sensor and check if a flower is detected
for index,ray in enumerate(zip(*self.rc_sensor.sensor_rays)):
     if ray[Sensors.RayCastSensor.HIT] and
        ray[Sensors.RayCastSensor.OBJECT_INFO]['tag'] == 'AlienFlower':
         print(f“Flower detected in ray: {index}”)
         print(ray)
```

Perception: Class InternalState [Aagent_BT.py]

- You access the information of the internal State of the agent through the property **`self.i_state`**.

```
isRotatingRight: <bool>
isRotatingLeft: <bool>
movingForwards: <bool>
movingBackwards: <bool>
isFrozen: <bool> Indicates if the agent is frozen due to a collision with an enemy
speed: <float> Current speed of the agent
position: <dict { "x": <float>, "y": <float>, "z": <float> } Position using world coordinates
rotation: <dict { "x": <float>, "y": <float>, "z": <float> } Rotation y - Yaw, x - Pitch, z - Roll
currentNamedLoc: <str> Name of the current location (if the agent is in one)
onRoute: <bool> Is the agent moving toward a target using the NavMesh system?
targetNamedLoc: <str> Name of the target location (if the agent is going to one using the NavMesh system)
myInventoryList: <list> My current inventory list. Has the form [{'name': '', 'amount': 0}, {'name': '',
'amount': 0}, ...]
nearbyContainerInventory: <bool> Is there a nearby container?
nearbyContainerInventoryList: <list> Nearby container inventory list.
        Has the form [{'name': '', 'amount': 0}, {'name': '', 'amount': 0}, ...]
```

The inventory

- The Astronaut has the possibility to store collectable objects. When the Astronaut is near enough to a collectable object, this is added to the Astronaut inventory, that is shown at the bottom of the screen.

- In the “AAC” scenario the only collectable objects are the alien flowers (tagged as “AlienFlower”).

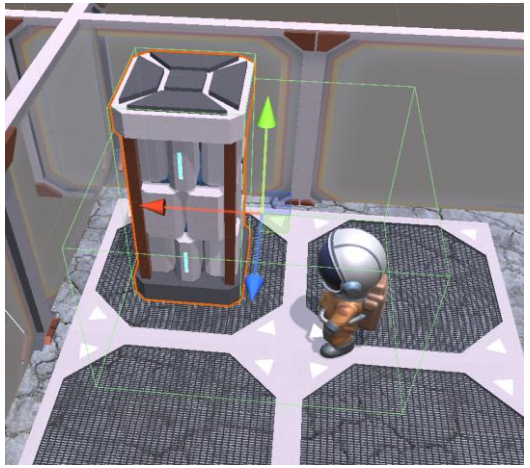
Alien flower

Astronaut’s Inventory

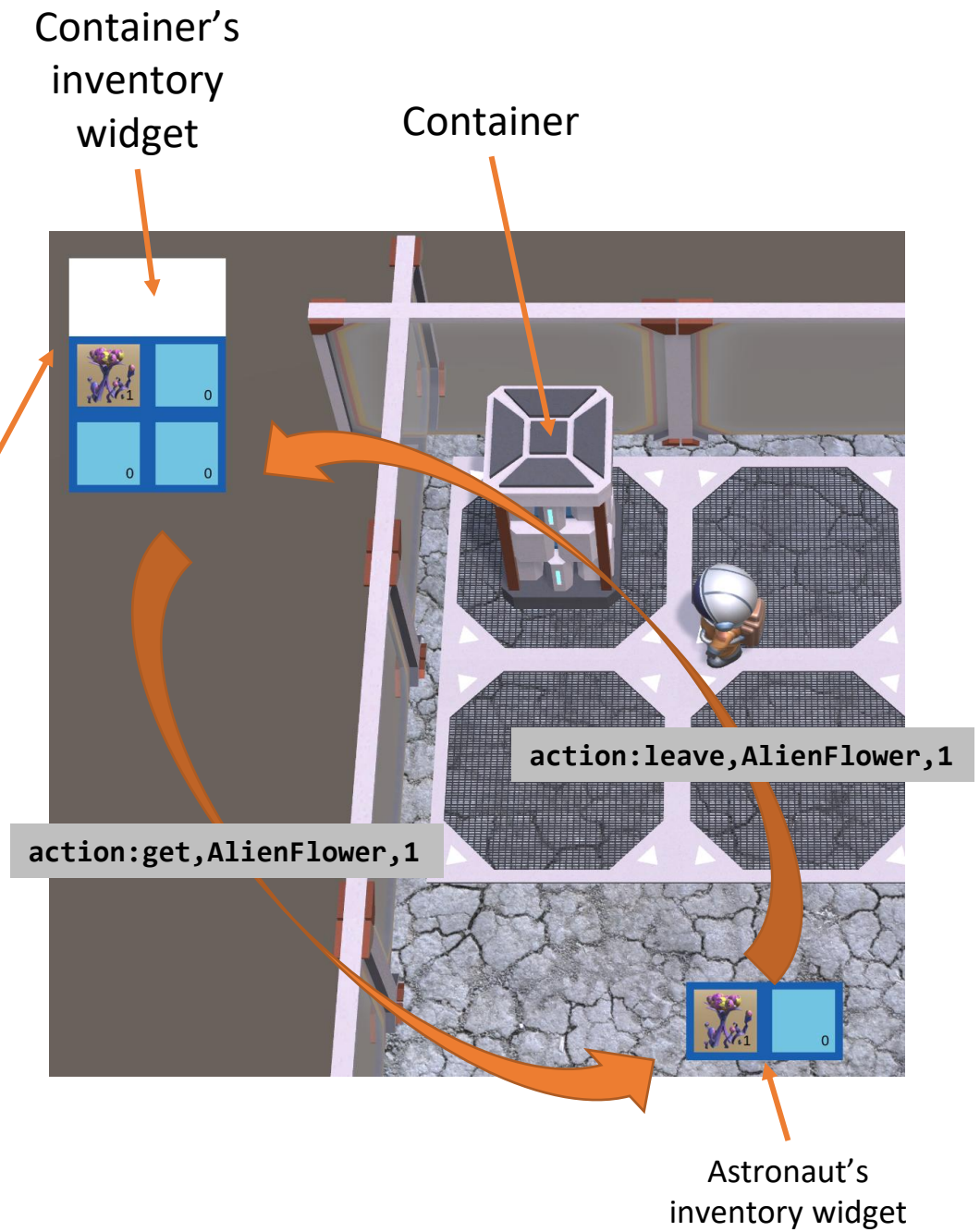
(In this screenshot, the astronaut has already collected one alien flower. Moving forward she will collect another one)

The inventory

- Some elements in the scene can also store objects. When the Astronaut approaches them, a new UI-widget appears showing what is inside that container and indicating that the Astronaut can “get” or “leave” objects that she already has in her inventory.



Container and its area of influence (not visible when AAPE is running). If the Astronaut is inside the area of influence of the container, the container’s inventory widget appears and the Astronaut can get and leave objects from/to that container using the corresponding actions.

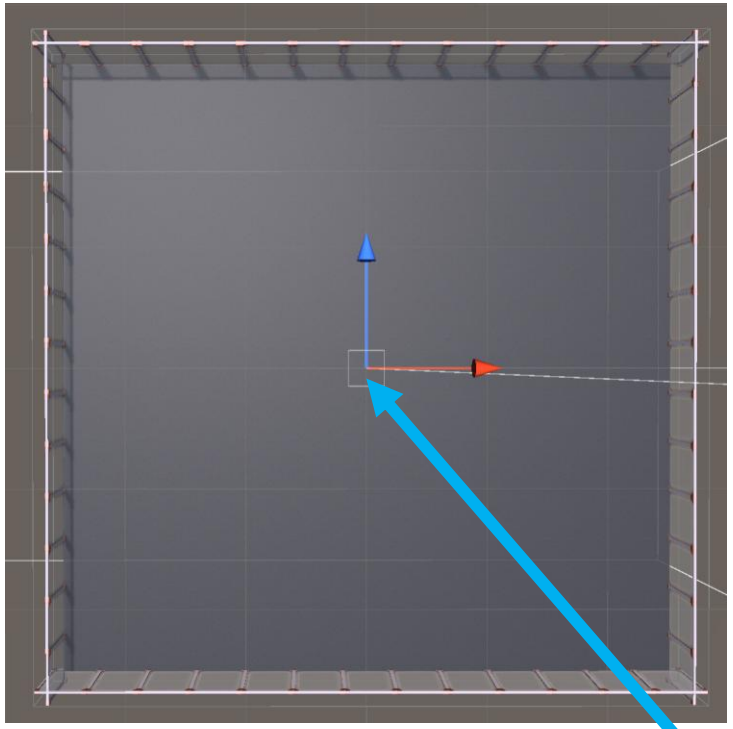


Examples:

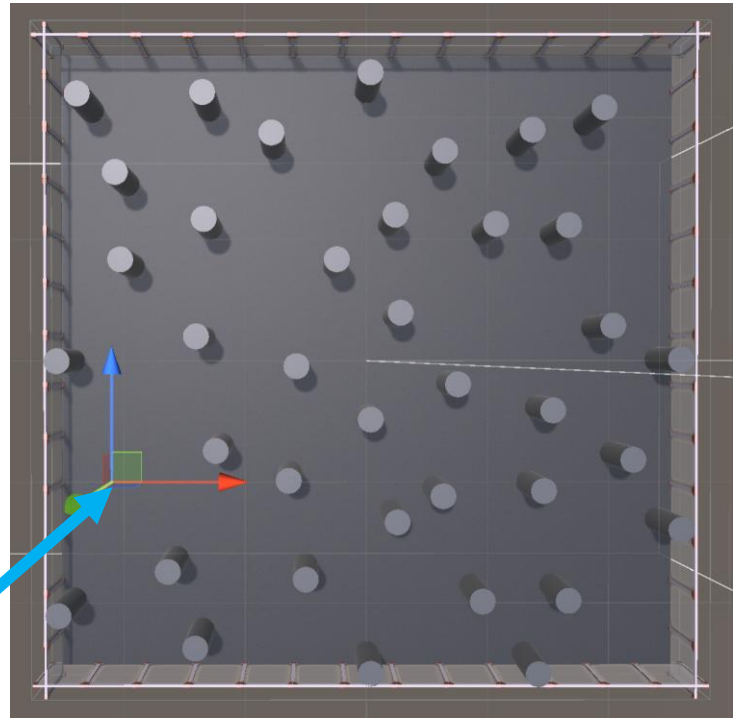
```
# Get the amount of elements of type ‘AlienFlower’
alien_flower_amount = next(
                (item['amount'] for item in self.i_state.myInventoryList if item['name'] == 'AlienFlower'),
                0  # You can use None or 0 if not found
            )
print(alien_flower_amount)
```

The Scenarios

“1-Empty” scene



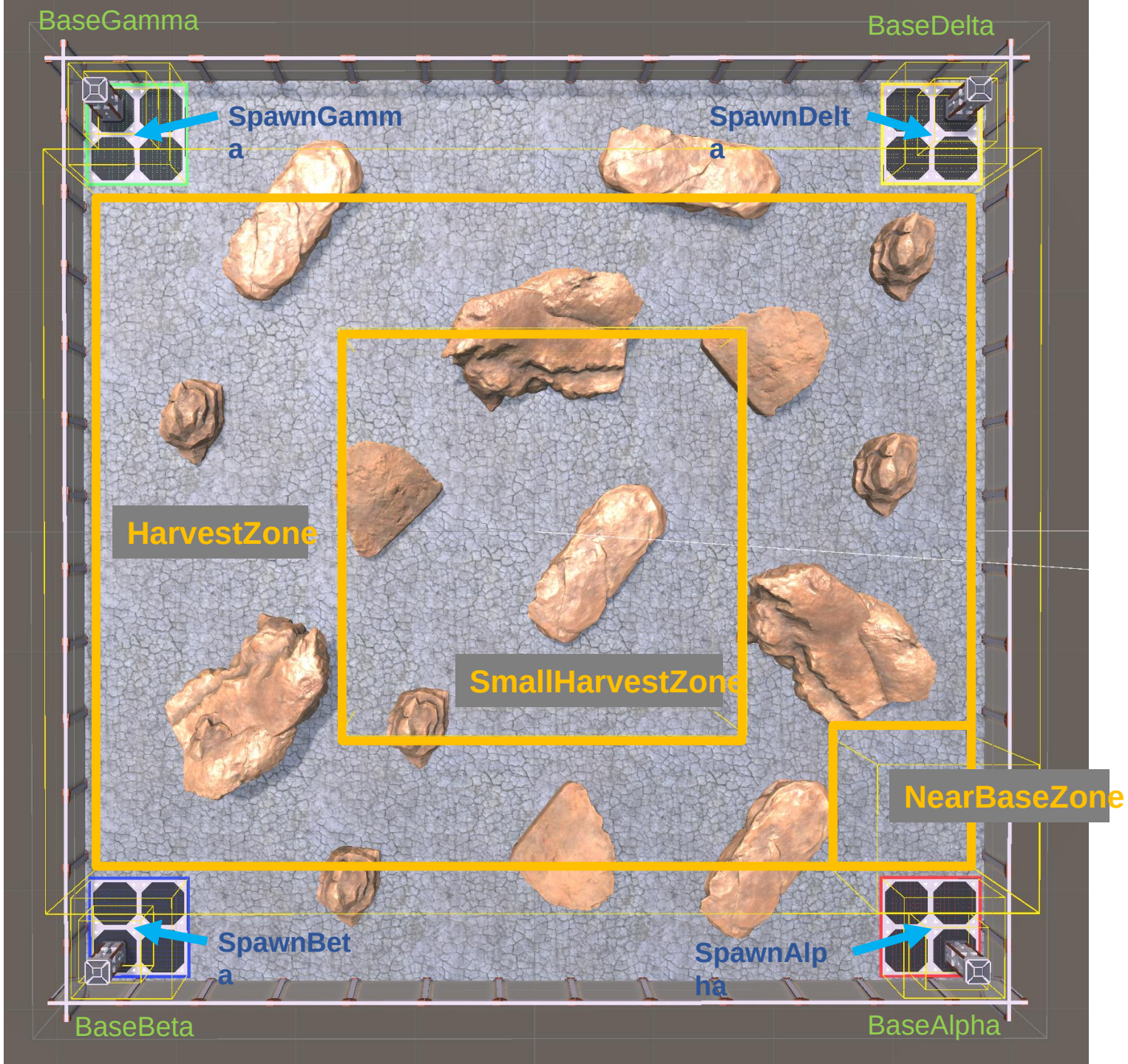
“2-Avoid” scene



SpawnPoint0

The Scenarios

“3-AAC” scene



Locations:
BaseAlpha
BaseBeta
BaseGamma
BaseDelta

Spawn areas:
HarvestZone
SmallHarvestZone
NearBaseZone

SpawnPoints:
SpawnAlpha
SpawnBeta
SpawnGamma
SpawnDelta

Agent files

Files:

AAgent_BT.py – Is the agent class. It instantiates the agent, deals with the connection with Unity and executes the main loop.

AAgent-X.json – Agent configuration files. One configuration file for each agent.

APackCritters.json – Agent pack configuration file example. Allows to spawn many agents at once.

Sensors.py – Deals with the information related to the sensors.

Goals_BT.py – File where the goals of the agent are implemented.

Here we will put the code we do
In this two scripts only

BTRoam.py – Example file with the implementation of a behaviour tree.

Spawner.py – Python script to spawn many agents at once.

Agent configuration file

AAgent-1.json

```
{
  "Server": {
    "host": "127.0.0.1",
    "port": 4649
  },
  "AgentParameters": {
    "type": "AAgentAstronaut",
    "name": "Astronaut1",
    "interactable": true,
    "spawn_area": "",
    "spawn_point":0,
    "spawn_distance": 2,
    "debug_mode": false,
    "manual_control": true,
    "initial_task": "",
    "ray_perception_sensor_param": [2,45,0,5]
  },
  "Misc": {
    "python_gui_monitor": true
  }
}
```

Websocket Connecion parameters (do not Touch)

Name

Unity Prefab

If the agent appears in the slot section or as a background NPC

Spawn area where the agent will appear in a random position

Spawn point where the agent will appear

Not used

Start with debug mode active?

Start with manual control active?

Task we want the agent to perform after loading and after starting from an on-hold state the simulation.

Sensor configuration

GUI with debugging information

Initial Task
A activity that will the agent do when the simulation starts
- Not very useful for actions
- Useful for btrees

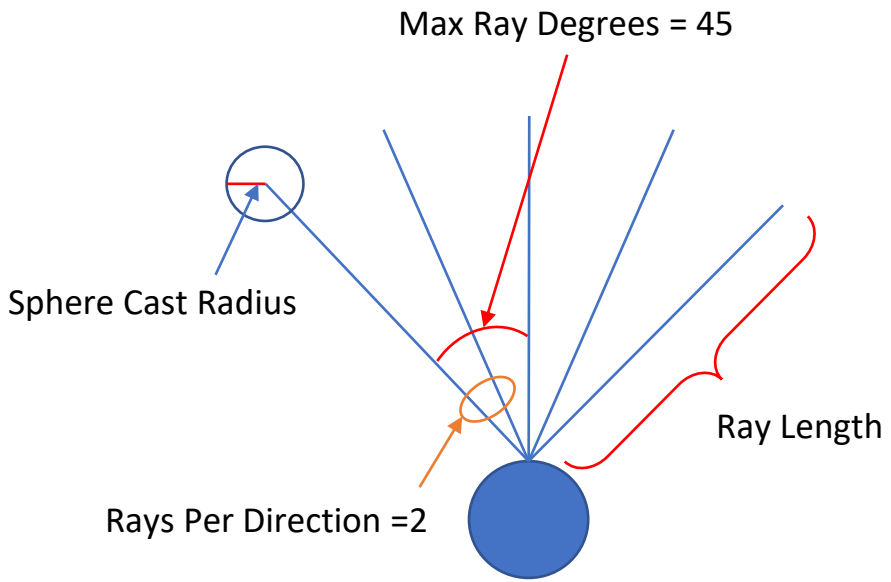
Agent configuration file

```
{
  "Server": {
    "host": "127.0.0.1",
    "port": 4649
  },
  "AgentParameters": {
    "name": "Drone1",
    "type": “AAgentDrone",
    "interactable": true,
    "spawn_area": "",
    "spawn_point": 0,
    "spawn_distance": 2,
    "debug_mode": false,
    "manual_control": false,
    "initial_task": "",
    "ray_perception_sensor_param":
[2,45,0,5]
  },
  "Misc": {
    "python_gui_monitor": false
  }
}
```

RAYS
Not seomthing we should touch MAYBE

```
[<Rays Per Direction>, <Max Ray Degrees>, <Sphere Cast Radius>, <Ray Length>]
```

<Rays Per Direction>: Number of rays on each side of the central one.
<Max Ray Degrees>: Angle between the central ray and the leftmost or righmost ray.
<Sphere Cast Radius>: Size of the sphere at the end of the ray.
<Ray Length>: Length of the rays



How to start a single agent

1. Execute the AAPE environment.
2. In a terminal, activate the python virtual environment.
3. Go to the “AAgent_Python” directory and run:
   `python Aagent_BT.py AAgent-X.json`
4. After a while, you will see the agent appearing in the environment.

Repeat for each agent you want to start.

The GUI debugging window

```
"Misc": {
  "python_gui_monitor": true
}
```

When an agent is instantiated with this parameter set to true, it will display a window showing the details about its internal state and the sensor.

AAGENT: Astronaut

```
[0, 0, None]
[1, 0, None]
[2, 0, None]
[3, 1, {'name': 'VegetationSmall04 (1)', 'tag': 'AlienFlower', 'distance': 4.189768}]
[4, 1, {'name': 'VegetationSmall04 (1)', 'tag': 'AlienFlower', 'distance': 4.207105}]
```

self.rc_sensor

```
isRotatingRight: False
isRotatingLeft: False
movingForwards: False
movingBackwards: False
isFrozen: False
currentNamedLoc: Base
onRoute: False
targetNamedLoc:
speed: 0.171756059
position: {'x': 30.2912788, 'y': 0.371745259, 'z': -30.6514626}
rotation: {'x': 0.0, 'y': 301.626282, 'z': 0.0}
myInventoryList:
{'name': '', 'amount': 0}
{'name': '', 'amount': 0}
nearbyContainerInventory: False
nearbyContainerInventoryList:
```

self.i_state

How to start a pack of agents

You can start a pack of agents at once using the “Spawner.py” tool. The spawner will start the agents specified in the json file that you pass as a parameter.

A pack configuration file consists of a list of packs. Each pack is composed by the name of an agent config file and the number of agents that the spawner will instantiate using that agent config file.

For example:

APackAstroCritters.json

```
{
    "packs": [
        {
            "agent_config_file": "AAgent-1.json",
            "num_agents": 1
        }
        {
            "agent_config_file": "AAgent-2.json",
            "num_agents": 10
        }

    ]
}
```

Example file that will start 1 agent of type AAgent-1.json and 10 agents of type AAgent-2.json

How to start a pack of agents

The pack configuration file consists of a list of packs. Each pack is composed by the name of an agent config file and the number of agents that the spawner will instantiate using that agent config file.

For example:

APackAstroCritters.json

```
{
    "packs": [
        {
            "agent_config_file": "AAgent-1.json",
            "num_agents": 1
        }
        {
            "agent_config_file": "AAgent-2.json",
            "num_agents": 10
        }

    ]
}
```

Example file that will start 1 agent of type AAgent-1.json and 10 agents of type AAgent-2.json

How to start a pack of agents

1. Execute the AAPE environment.
2. In a terminal, activate the python virtual environment.
3. Go to the “AAgent_Python” directory and run:
   ```
   python Spawner.py <APackFileConfig.json>
   ```
4. After a while, you will see the agents appearing in the environment.

Repeat for each agent you want to start.

```
{
  "Server": {
    "host": "127.0.0.1",
    "port": 4649
  },
  "AgentParameters": {
    "name": "Drone1",
    "type": “AAgentDrone",
    "interactable": true,
    "spawn_area": "",
    "spawn_point": 0,
    "spawn_distance": 2,
    "debug_mode": false,
    "manual_control": false,
    "initial_task": "",
    "ray_perception_sensor_param":
[2,45,0,5]
  },
  "Misc": {
    "python_gui_monitor": false
  }
}
```

There are several agent parameters that you probably want to use when starting a pack of agents:

“interactable”: false

Interactable true means that the agent will appear in the agent slots and that you can, for instance, control it manually. There are only 8 free slots so is better to reserve them for the important agents. Interactable false means the agent is running as an NPC and you have no direct access to it in the GUI. So when you start a pack of 10 critters you probably want to have interactable = false.

“spawn_area”: “<name_of_the_area>”

In the AAPE scenarios sometimes there are areas where you can spawn agents. They are spawned randomly distributed. The alternative is to use a **“spawn_point”**. If you start may agents using Spawner.py, use a spawn_area. When you spawn a single agent, you can use a spawn_point or also a spawn_area.

```
{
  "Server": {
    "host": "127.0.0.1",
    "port": 4649
  },
  "AgentParameters": {
    "name": "Drone1",
    "type": “AAgentDrone",
    "interactable": true,
    "spawn_area": "",
    "spawn_point": 0,
    "spawn_distance": 2,
    "debug_mode": false,
    "manual_control": false,
    "initial_task": "",
    "ray_perception_sensor_param":
[2,45,0,5]
  },
  "Misc": {
    "python_gui_monitor": false
  }
}
```

There are several agent parameters that you probably want to use when starting a pack of agents:

“debug_mode”: false
“manual_control”: false
This two options activate the raycast debug mode and the manual control. Because the agents in a pack are usually not interactable, you will not be able to change this options once the agents appear in the scene. Therefore, you probably will want this two options to be false for agents in a pack (specially “manual_control”).

```
{
  "Server": {
    "host": "127.0.0.1",
    "port": 4649
  },
  "AgentParameters": {
    "name": "Drone1",
    "type": “AAgentDrone",
    "interactable": true,
    "spawn_area": "",
    "spawn_point": 0,
    "spawn_distance": 2,
    "debug_mode": false,
    "manual_control": false,
    "initial_task": "",
    "ray_perception_sensor_param":
[2,45,0,5]
  },
  "Misc": {
    "python_gui_monitor": false
  }
}
```

There are several agent parameters that you probably want to use when starting a pack of agents:

“initial_task”: “<action|goal|bt>”

The initial_task is the **action|goal|bt** that the agent will execute automatically once it appears in the scene. It can be used for a single agent but when it is really useful is when you want to specify an initial_task for a pack of agents. For example you could use “initial_task”: “bt:BTRoam” in a AAgent-X.json configuration file and the use Spawner.py to initiate 10 agents using AAgent-X.json. This will instantiate 10 agents that will run the behaviour tree BTRoam as soon as the simulation starts running. This initial_task is also executed when starting the simulation again after an on-hold state.

“python_gui_monitor”: false

If this parameter is true, it will show the agent debugging GUI screen. This window shows the information about the agent internal state. Probably you will want this option to be deactivated for the agents in a pack to avoid .

Take a look at
AAgent_BT.py

Programming Basic Behaviors

Behaviors

```
self.a_agent -> Reference to the AAgent parent class.
self.a_agent.rc_sensor-> Access to the ray cast sensor information (reference to
                         an object of type RayCastSensor defined in Sensors.py).
self.a_agent.i_state -> Access to the internal state of the agent.
```

self.a_agent will be used to call the send_message method. Because in this part of the code we are requesting for actions, the call will have the form:

```
await self.a_agent.send_message("action", "<action_to_perform>")
```

Where `<action_to_perform>` is a string with one of the possible actions [tl, tr, nt, mf, mb, stop].

For example,

To move forward you will send the message **`self.a_agent.send_message("action","mf")`** to turn to the right **`self.a_agent.send_message("action","tr")`** and so on. **You can pass a single action per message.**

“DoNothing”

Behavior: “Do nothing”

```
class DoNothing:
    """
    Does nothing
    """
    def __init__(self, a_agent):
        self.a_agent = a_agent
        self.rc_sensor = a_agent.rc_sensor
        self.i_state = a_agent.i_state

    async def run(self):
        print("Doing nothing")
        await asyncio.sleep(1)
        return True
```

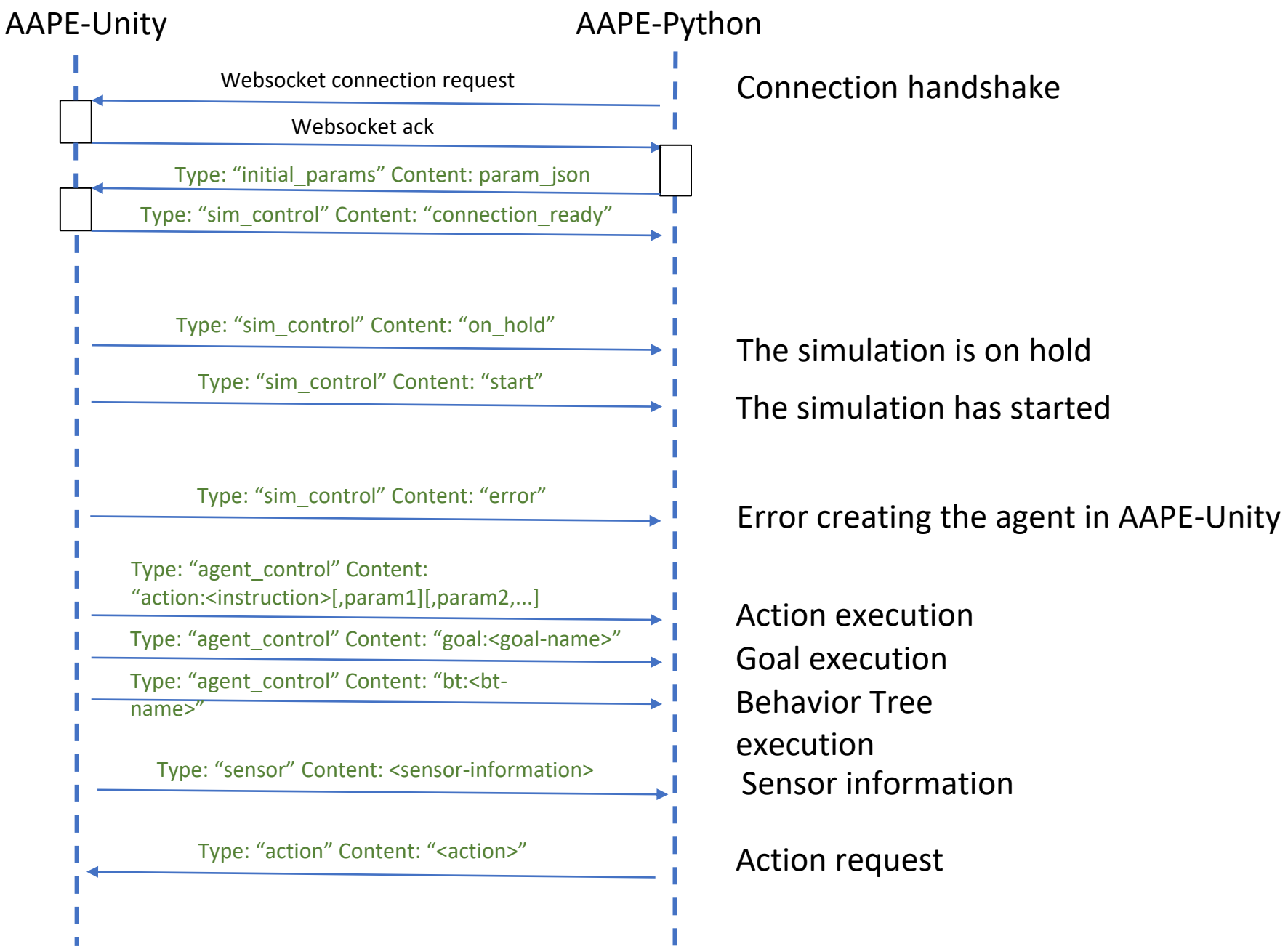
“ForwardStop”

Behavior: “Start moving forward till you find an obstacle and then stop”

```
class ForwardStop:
    STOPPED = 0
    MOVING = 1
    END = 2

    def __init__(self, a_agent):
        self.a_agent = a_agent
        self.rc_sensor = a_agent.rc_sensor
        self.i_state = a_agent.i_state
        self.state = self.STOPPED

    async def run(self):
        try:
            while True:
                if self.state == self.STOPPED:
                    # Start moving
                    await self.a_agent.send_message("action", "mf")
                    self.state = self.MOVING
                elif self.state == self.MOVING:
                    sensor_hits = self.rc_sensor.sensor_rays[Sensors.RayCastSensor.HIT]
                    if any(ray_hit == 1 for ray_hit in sensor_hits):
                        self.state = self.END
                        await self.a_agent.send_message("action", “ntm")
                    else:
                        await asyncio.sleep(0)
                elif self.state == self.END:
                    break
                else:
                    print("Unknown state: " + str(self.state))
                    return False
        except asyncio.CancelledError:
            print("***** TASK Forward CANCELLED")
            await self.a_agent.send_message("action", "stop")
            self.state = self.STOPPED
```

Appendix

Interaction diagram